

14 Зэрэгцээ программчлал

14.1 Урсгал ба процесс

Программыг биелүүлэх явцад, хяналтад хамрагдсан бүх командыг биелэх дарааллын дагуу холбон программын текст бүтцийн хувьд *урсгал* гэж төсөөлөн хяналтын бүдүүвчийг үр ашигтайгаар зураглаж болно. Хяналтын урсгал буюу үндсэндээ урсгал нь тусгай тооцоололд биелсэн программын командуудын дарааллыг тодорхойлдог. *Дараалсан* программын хувьд ажиллах явцад идэвхтэй ганц урсгал байдаг бол *зэрэгцээ* программд хэд хэдэн идэвхтэй урсгал байдаг. Ихэнхдээ урсгалуудын оронд хүмүүс процесс, даалгавар эсвэл зэрэгцээ программын тухай ярьдаг боловч гэхдээ нэгэн зэрэг идэвхтэй байх тусдаа ажилладаг хэд хэдэн контексттэй тохиолдлыг тэд авч үздэг. Мөн хэлний түвшинд нэр томъёо нь нэгдмэл байдаггүй, учир нь зарим хэл нь бидний нэрлэсэн зүйлийг урсгал гэж нэрлэдэг бол зарим нь (жишээ нь Ада) тэдгээрийг даалгавар гэж нэрлэдэг.

Энд, бид нэгэн хэлбэрийн нэр томъёог ашиглахыг хичээх бөгөөд дээр дурдсан утгаараа урсгал гэсэн нэр томъёог ашиглах бөгөөд ерөнхийдөө үйлдлийн системд байдагтай адил хувийн хаягийн орон зай бүхий биелэгдэх зааврын нийтлэг багцыг илэрхийлэхийн тулд *процесс* гэдэг нэр томъёог ашиглана.¹ Энэ талаас нь бид процессыг нөөц (жишээ нь, санах ой) хуваалцаж буй хэд хэдэн урсгалаас бүрдсэн гэж харж болно.² Ерөнхийдөө үйлдлийн систем нь хувийн хаягийн орон зай бүхий хүнд жинтэй процессуудыг дундын орон зайг хуваалцаж болох хөнгөн процессоос ялгадаг. Энэ нэр томъёог заримдаа урсгалд ашигладаг; энд бид үргэлж хөнгөн урсгалыг авч үзнэ.

14.2 Түүхэн товчоо

Компьютерийн шинжлэх ухааны бусад олон чухал чиглэлүүдийн нэгэн адил зэрэгцээ байдал нь техник хангамжийн архитектурын шинэчлэлээс үүдэлтэй: 1960-аад онд device controller нэвтрүүлсэн. Эдгээр нь дагалдах төхөөрөмжид CPU-ийн шүүд оролцоогүйгээр өгөгдлийн оролт, гаралтыг бие даан удирдах боломжийг олгодог техник хангамжийн механизмууд юм. Эдгээр төхөөрөмжүүд гарч ирэхээс өмнө гадаад төхөөрөмжийн удирдлага маш үр ашиггүй байсан: CPU нь оролт, гаралтын үйлдлийг гүйцэтгэх командыг дагалдах төхөөрөмж рүү илгээж, дараа нь дуустал хүлээнэ. Дагалдах төхөөрөмж нь харьцангуй (жишээ нь санах ойн регистрээс илүү) удаан ажилладаг байсан тул дагалдах төхөөрөмжид хандах нь дагалдах төхөөрөмжөөс өгөгдөл өгөх/авахыг хүлээхэд CPU-ийн циклийн тодорхой хэсгийг дэмий үрдэг гэсэн үг юм (нэг карт уншигч секундэд дөрвөн карт уншиж чаддаг байсан бол жараад оны компьютер л секундэд дөчин мянга орчим зааврыг гүйцэтгэж чаддаг байсан гэж бодоорой). *Тасалдал* дээр суурилсан төхөөрөмжийн контроллер ба тэдгээрийн харилцааны механизм нь энэхүү үр ашиггүй байдлыг шийдсэн: одоо CPU нь оролт эсвэл гаралтын үйлдлийг гүйцэтгэхийг хүсч байвал зохих төхөөрөмжийн хянагч руу команд илгээж, дараа нь бусад зааврын гүйцэтгэлийг үргэлжлүүлнэ. Дагалдах төхөөрөмж ажиллаж дуусмагц контроллер нь CPU-г тасалдуулах дохио өгдөг. Энэ дохио нь дагалдах төхөөрөмж рүү илгээсэн командыг дууссан, боловсруулахад зориулагдсан өгөгдөл байгааг CPU-д хэлдэг. Ихэвчлэн (боломжтой бол) CPU нь эдгээр дохиог хүлээн авах үед одоогийн зааврын гүйцэтгэлийг тасалдуулж, харгалзах тасалдал

¹ Зэрэгцээ байдлын онол болон "процесс алгебр" гэж нэрлэгддэг салбарт "процесс" гэсэн нэр томъёо нь ихэвчлэн бодит программтай тодорхойлогддог хийсвэр тооцооллын нэгжийг илэрхийлдэг болохыг анхаарна уу.

² Гэсэн хэдий ч хэрэгжилт талаас нь авч үзвэл үйлдлийн системийн цөмийн командуудыг ашиглан процессуудыг удирдах боломжтой.

баригчаар дамжуулан дохиотой харьцдаг (жишээ нь, дагалдах төхөөрөмжөөс уншсан өгөгдөл дээр хэрхэн ажиллахыг CPU-д зааварладаг).

Тасалдал нь CPU болон дагалдах төхөөрөмжийн харилцан үйлчлэлийг зэрэгцээ, илүү үр ашигтай байдлыг бий болгох бөгөөд програмчлалд шинэ асуудлуудыг оруулж, улмаар судлаачдыг зэрэгцээ байдлыг удирдах механизмыг судлахад хүргэсэн. Ялангуяа зэрэгцсэн байдал нь *уралдааны нөхцөлийг* үүсгэдэг, өөрөөр хэлбэл, тооцооллын үр дүн нь үйлдлийн харьцангуй эрэмбээс шалтгаалж дундын өгөгдөлд хандах үед тохиолддог нөхцөл байдлыг үүсгэдэг. Жишээлбэл, дискнээс өгөгдөл ашиглан тооцоолол хийдэг програмыг төсөөлөөд үз дээ. Программ дагалдах төхөөрөмжөөс өгөгдлийг унших явцад бусад утга дээр тооцооллоо үргэлжүүлдэг байхыг хүснэ. Гэсэн хэдий ч дискний дохио хэзээ тооцооллыг тасалдуулахыг бид таамаглаж чадахгүй: Тасалдал нь санах ойд зарим утгыг ачаалсны дараа тооцооллыг зогсоож, улмаар тасалдал баригч эдгээр утгыг дарж бичиж, тооцоолол дахин үргэлжлэх үед программ бүрүү өгөгдөл дээр ажиллаж болно.

Ерөнхийдөө, зэрэгцээ байдал нь өөр өөрөөр биелэгдэх үр дүнд хүргэж болзошгүй эрэмбийн хослолоор зэрэгцсэн зааврыг гүйцэтгэх боломжийг CPU-д олгодог. Үүнийг илүү сайн ойлгохын тулд дээр зурагласан CPU-дагалдах төхөөрөмж зохиомжийн энгийн жишээг (гэхдээ агуулгын хувьд ижил) авч үзье. Бидний хялбаршуулсан жишээн дээр бид программ T 1 ба T 2 гэсэн хоёр урсгалтай бөгөөд хоёулаа дундын х хувьсагчийн утгыг нэмэгдүүлдэг, эхлээд үүнийг үндсэн санах ойноос регистр рүү хуулна. T1 ба T2-ын биелэлтийн эрэмбэд нэмэлт шаардлага байхгүйгээр биелэлт зөв байна (prefixing each action with the thread that executes it).

```
T1 | read x in reg1
T2 | read x in reg2
T2 | increment reg2
T2 | write reg2 in x
T1 | increment reg1
T1 | write reg1 in x
```

Дээр гарсан асуудал бол T1 нь T2-ын х дээр үзүүлэх өөрчлөлтийг хүчингүй болгож байна. Х нь 0 утгыг агуулж байна гэж үзье. T 1 нь х-ийн утгыг уншдаг бөгөөд энэ нь 0. Дараа нь T 2 нь х-ийн 0 утгыг уншдаг бөгөөд үүнийг 1 хүртэл нэмэгдүүлж, түүнийг х дээр бичнэ. Эцэст нь T1 нь х-ийн уншилтыг 1 болгон нэмэгдүүлж, х дээр бичнэ. Мэдээжийн хэрэг, хэрэв бид жишээлбэл, T1 ба T2 нь ар араас нь биелдэг байсан бол х нь 1 биш харин 2 гэсэн утгыг агуулна. Дээрх зохиомж нь T1 ба T2-ын зааврын зөвшөөрөгдөх 20 гаруй дарааллаас ердөө хоёр нь бөгөөд эдгээрээс зөвхөн хоёр нь х утгыг 2 болгоно - T1 эсвэл T2-ийг бүтэн, дараа нь нөгөө урсгалыг бүтэн гүйцэтгэсэн тохиолдлууд.

Харж байгаачлан тооцооллыг дагаж мөрдөж болох замуудын тооноос шалтгаалан зэрэгцсэн жижиг программын ч ажиллагааны талаар дүгнэлт хийхэд хэцүү байдаг. Зэрэгцээ тооцооллыг урьдчилан таамаглах нэг арга бол өөр өөр үйлдлийн хооронд синхрончлолыг нэвтрүүлэх юм. Энэ нь тасалдлыг ашиглах, нэвтрүүлэхтэй утга дүйцэх харилцан хориглох (mutual exclusion) механизм, эгзэгтэй муж (critical section) зэрэг синхрончлал болон зэрэгцээ байдлыг удирдах анхны хэрэгслүүд танилцуулагдсан явдал байна. Багц (batch) хэлбэрийн анхны мултипрограмчлалын системүүд нь зөвхөн үйлдлийн системийн түвшинд маш хязгаарлагдмал хэлбэрээр зэрэгцэн ажиллахыг зөвшөөрдөг (нэг процесс нь зөвхөн оролт, гаралтын үйлдлээр л CPU руу хандах эрхийг бусад процесст шилжүүлдэг). Дараа нь, time-sharing систем бий болсноор хэрэглээний түвшинд зэрэгцэн

биелэх ажиллагаа гарч, ингэснээр дээд түвшний хэл дээр харилцаа болон синхрончлолын бүтэц орсон.

60-аад оны сүүлчээр мультипроцессорын анхны архитектурууд хэрэгжиж эхэлснээр зэрэгцээ ажиллах техник, моделийг хөгжүүлэх хоёр дахь чухал түлхэц болсон юм. Эдгээр архитектурт хэд хэдэн процессор байгаа нь өөр өөр програмууд эсвэл нэг програмын өөр өөр хэсгүүдийг хамтран (эсвэл параллел) гүйцэтгэхэд тусалдаг. Энэ боломж нь алгоритмын болон арга зүйн шинэ асуудлуудыг бий болгосон: Үнэндээ, *параллел программчлал* нь програмчлалын хэлтэй ихэвчлэн холбоотой байдгаас салбаруудыг хамардаг шинжлэх ухааны тооцооллын хэрэглээнд зориулж боловсруулагдсан.

Зэрэгцээ програмчлалыг хөгжүүлэх гурав дахь чухал элемент нь 1970-аад онд анхны компьютерийн сүлжээнүүд бий болсон явдал. Өөр, өөр машинууд бие биенээсээ хол зайд байсан ч нэг нь нөгөөдөө нээсэн замаар “ярилцдаг” зэрэгцээ байдлын өөр нэг модел: *хуваарилагдсан тооцоолол*, өөр өөр машинууд дээр ажилладаг програмууд нь холбооны тохиромжтой дэд бүтцээр дамжих мессеж солилцоонд суурилсан механизмуудыг ашиглан хамтран ажиллах боломжтой.

14.3 Зэрэгцээ программчлалын төрлүүд

Дээрх товч түүхэн тэмдэглэлээс харахад бид олон төрлийн зэрэгцээ програмчлалыг ялгаж болно.

Анхлан суралцагчдад зориулж бид *физик зэрэгцээ байдал (physical concurrency)* -ыг *логик зэрэгцээ байдал (logical concurrency)*-аас ялгана. Эхний тохиолдолд, жишээ нь хэд хэдэн процессортой машинуудын хувьд хоёр үрсгалын (эсвэл процесс, эсвэл бүр програмуудыг) биелэлт хамтран эсвэл параллел байна. Хоёрдахь тохиолдолд, бид зөвхөн логик түвшний зэрэгцээ байдалтай, өөрөөр хэлбэл программист болон программ хангамж нь биелэлтийг зэрэгцээ гэж үздэг боловч физикийн хувьд зэрэгцээ ажиллагаа нь дараалсан биелэлтэнд задардаг. Энэ нь мультипрограмчлалд нэг процессортой машиныг нэгэн зэрэг ашигладаг хэрэглэгчдийн тохиолдол юм. Програмчлалын хэлнүүдийн үзэл баримтлалын хувьд энэ ялгаа нь системийн гүйцэтгэлийн үзүүлэлт, биелэх хугацааг тооцохоос бусад тохиолдолд тийм ч чухал ач холбогдолтой биш – өөрөөр хэлбэл, хэрэв бид хугацааны T_s нэгжид ажилладаг тухайн асуудлын дараалсан шийдэлтэй, мөн ижил бодлогыг T_p -д ажилладаг зэрэгцээ шийдэлтэй бол физик зэрэгцүүлэлтийн хувьд $T_p < T_s$ байх болно.

Параллел программчлал

Физик зэрэгцээ програмчлалыг ашигладаг програмчлалын төрлийг ерөнхийдөө *параллел програмчлал* гэж нэрлэдэг бөгөөд энэ нь асуудлын үр ашигтай зэрэгцээ шийдлийг тодорхойлоход шаардлагатай алгоритмын болон арга зүйн асуудлуудыг агуулдаг. Энэ талбарт параллелизмын үндсэн эх сурвалж нь нэг үйлдлийг өөр өөр өгөгдөлд хэрэглэснээс үүсэлтэй *өгөгдлийн параллелизм* ба ижил өгөгдлийн багцад өөр өөр үйлдлүүдийг параллел ашигладаг *үйлдлийн параллелизмыг* гэж ялгана. Хоёр төрлийн параллелизм нь вектор процессор гэх мэт тусгай мультипроцессор архитектур ашигладаг, шинжлэх ухааны тооцооллын салбарт чухал практик хэрэглээтэй байдаг. Параллел програмчлалын энгийн хэрэглээ нь цаг агаарын модел гэх мэт дифференциал тэгшитгэлийн шийдэлд хамаатай бөгөөд матрицын тооцоололд суурилсан давталтын аргуудыг систем тэгшитгэлийг шийдвэрлэхэд үр ашигтай параллелчлах боломжтой. Өөр өөр процессор дээрх ажлын хуваалт нь уг шийдэлд ашигласан алгоритм болон програмчлалын аргууд процессоруудын дээрх ачаалал тэнцвэржүүлэлтийг дэмждэг тохиолдолд гүйцэтгэлийг мэдэгдэхүйц нэмэгдүүлэхэд хүргэдэг.

Параллел програмчлал нь тусгай алгоритм, арга зүйн асуудлуудыг танилцуулж байгаа хэдий ч хэл шинжлэлийн үүднээс энэ нь логик зэрэгцээ байдлаас онцгой ялгарах асуудал байхгүй. Үнэндээ параллел програм нь дундын санах ойтой мультипроцессор машинуудаас мультикомпьютер³ буюу тусдаа санах ойтой компьютер сүлжээ хүртэлх янз бүрийн суурь архитектур ашиглаж болно. Харилцаа, синхрончлолын хувьд нэг ийм програм нь логик зэрэгцээ байдлын хүрээнд бидний харсан ижил энгийн механизмуудыг (дундын санах ой, мессеж солилцоо) ашиглаж болох бөгөөд энэ шалтгааны улмаас бид сүүлийнх дээр голчлон анхаарч байна.

Цаашид зүгээр л зэрэгцээ гэж нэрлэгдэх *логик зэрэгцээ байдлын* хүрээнд дотор нь бид *мультиурсгалт програмчлал* ба *хуваарилагдсан програмчлал* гэж ялгана.

Мультиурсгалт программчлал

Бид дээр мультиурсгалтай програмчлалын тухай ойлголтыг танилцуулсан бөгөөд энэ нь нэгээс олон үрсгал эсвэл процесс байхтай холбоотой юм. Тодруулбал, энэ нь нэг машин (нэг процессор эсвэл дундын санах ойн мультипроцессор) дээр ажиллаж буй нэг программын нэгээс олон идэвхтэй биелэлтийн контекст ижил цаг хугацаанд оршин байна. Зэрэгцсэн үрсгал ба процессуудыг ашиглах хэрэгцээ, шаардлага нь тухайн програмын шийдвэрлэх ёстой асуудлын мөн чанараас хамаарна. Жишээлбэл, орчин үеийн хөтчийг авч үзье: бид шинэ хуудас (мөн ихэвчлэн шинэ цонх) нээх холбоосоор орвол программ нь шинэ хуудастай холбоотой пакетуудын харилцааг удирдах, хэлбэржүүлэх (formatting) болон үзүүлэх (магадгүй шинэ үрсгалуудыг нээх) гэх мэт шинэ үрсгалыг (эсвэл бүр шинэ процесс) нээдэг. Үүний зэрэгцээ, хэрэглэгч удаан үйлдлийн (зураг харуулах гэх мэт) биелэлтээс болж илүү хурдан үйлдлийг шийтгэх эрсдэлгүйгээр хөтчийн бусад функцуудтай харилцах боломжтой. Мэдээжийн хэрэг, нэг процессортой машинд CPU-г шууд л эзэлдэг (жишээ нь, удаан ажиллагаа дуусахыг хүлээх) эсвэл хэт удаан (бусад тооцоолол үргэжлэхийг хориглоно) үрсгалаас зайлсхийхийн тулд хэрэгжүүлэх зарим арга техник хэрэгтэй.

Хуваарилагдсан программчлал

Хуваарилагдсан програмчлал гэдэг нэр томъёо нь тархсан физик бүтцээс бүрдсэн машинууд дээр ажиллах зориулалттай зэрэгцээ программуудыг бий болгохыг хэлдэг. Эдгээр нь тархсан санах ойтой мультикомпьютер байж болохоос гадна янз бүрийн архитектур, тополог бүхий компьютеруудын бодит сүлжээ байж болно. Энд хамгийн чухал зүйл бол тооцооллын системийн зангилаанууд нь физикийн хувьд тархсан, санах ойгоо хуваалцаж чадахгүй болохоор мессеж солилцоонд суурилсан зохих механизм, харилцааны командуудыг нэвтрүүлэх ёстой. 15-р бүлэг нь энэхүү сүүлийн зэрэгцээ хэлбэр, түүний сүүлийн үеийн хөгжлийн талаархи хэлэлцүүлгийг хэлний хүрээнд гүнзгийрүүлэхэд зориулагдсан болно.

Зэрэгцээ программчлалын бусад төрлүүд

Бид зэрэгцээ програмчлалын янз бүрийн талуудыг ялгаж салгаж болох хоёрдахь хэмжигдэхүүн нь зэрэгцээ идэвхжлийн нарийвчилсан байдалтай холбоотой: эдгээр нь доод түвшний заавар, дээд түвшний хэлний командууд, програмын хэсгүүд эсвэл бүхэлдээ тусдаа програмууд байж болно.

³ Тодорхой болгох үүднээс. Бид дундын санах ойтой машинуудын хувьд "мультипроцессор", хуваарилагдсан санах ойтой машинуудын хувьд "мультикомпьютер" гэсэн нэр томъёог ашигладаг. Энэ салбарын нэр томъёо нь нэгдмэл биш бөгөөд энэ сэдвээр олон хувилбар байж болно. Одоогийн хэлэлцүүлгийн хувьд өөр өөр процессоруудын хооронд дундын санах ой байгаа эсэх нь чухал.

Энэхүү шатлалын хоёр захын тохиолдол нь хэл шинжлэлийн тодорхой асуудлуудаас илүүтэй архитектурын асуудал, харилцааны протоколуудтай холбоотой асуудлуудтай холбоотой тул одоогийн хэлэлцүүлгийн зорилгод тийм ч сонирхолтой биш юм. Мөн нэгж-командын зэрэгцээ байдал нь практикт чухал ач холбогдолтой хэдий ч зэрэгцээ байдлыг удирдах ил бүтцээс илүү үр ашигтай параллел компиляцад хамаатай. Үнэндээ энэ төрлийн зэрэгцээ байдалд ихэвчлэн санах ойд өгөгдлийг хэрхэн хуваарилах, хэдэн процессор ашиглах гэх мэтийг компиляторт мэдээлдэг тусгай тэмдэглэгээ бүхий уламжлалт дараалсан хэлийг хамардаг. Энэ мэдээллийг ашиглан компилятор нь физик машины онцлогуудыг (ихэвчлэн хуваалцсан санах ойтой олон процессор) ашиглах боломжтой оновчилсон параллел объект кодыг гаргадаг. Энэ төрлийн параллел компиляцыг зөвшөөрдөг хэлний жишээ бол 90-ээд оны үед санал болгосон Фортран 90 хэлний өргөтгөл болох High Performance Fortran (HPF) бөгөөд өгөгдөл дээр параллелизм бүхий программыг бүтээх боломжтой, ялангуяа шинжлэх ухааны хэрэглээнд ашиглагддаг.

Тиймээс энэ бүлэгт бид дэд програм, нэгж гэх мэт програмын хэсгүүдийн түвшинд зэрэгцэн ажиллахад анхаарлаа хандуулна. Үнэндээ, энэ тохиолдолд бид хэсгүүдийн хоорондын харилцаа, синхрончлолыг удирдахын тулд тодорхой дээд түвшний хэл шинжлэлийн бүтэц хэрэгтэй байгааг бид дараагийн хэсэгт илүү нарийвчлан үзэх болно.

Нэр томъёоны тайлбар

14.7-р бүлэгт авч үзсэн Java-н тодорхой тохиолдлоос бусад дараах хэлэлцүүлэг нь урсгалын тохиолдол болон процессуудын хооронд онцгой ялгахгүй. Тиймээс, энгийн байх үүднээс 14.7-г хүртэл бид "процесс" гэсэн нэр томъёог "урсгал" гэсэн ижил утгатай болгон ашиглана.

14.3.1 Харилцах механизмууд

Зэрэгцсэн програмчлалын хүрээнд (төрлөөс үл хамааран) ялгаатай загварууд болон хэлнүүдийн хоёр нийтлэг тал нь *харилцааны, синхрончлолын* механизм юм.

Харилцааны механизмын хэрэгцээ нь тодорхой: функцийг хэрэгжүүлэхэд оролцдог янз бүрийн идэвхжлүүд бие биентэйгээ харилцах, олон төрлийн мэдээлэл солилцох (жишээлбэл, тооцооллын хэсэгчилсэн үр дүн эсвэл хяналтын харилцаа) арга зам хэрэгтэй. Энхүү механизмд харилцаж буй хоёр оролцогч талыг авч үздэг бөгөөд тэднийг үүргээр нь ялгаж болно: Нэг нь мэдээллийг үйлдвэрлэдэг, нөгөө нь хүлээн авдаг (хэрэглэдэг). Онолын хувьд бид харилцааны хоёр үндсэн механизмыг ялгадаг. Үйлдвэрлэгч ба хэрэглэгч нь хэрхэн хэрэгжиж байгаагаас хамааран харилцааны хоёр суурь механизмыг тодорхойлдог: дундын санах ойд суурилсан, мессеж солилцоонд суурилсан. Эдгээр хоёр төрөл нь үнэндээ бидний 14.4 ба 14.5-д нарийвчлан шинжлэх зэрэгцээ байдлын моделийн өөр өөр ангиллыг тодорхойлдог. Энд бид зөвхөн тэдний үндсэн шинж чанарыг дурдах сонирхолтой байна.

Blackboard

Дундын санах ой болон мессеж солилцоо хооронд завсрын харилцааны модель бол *самбар (blackboard)* архитектур (эсвэл дундын агуулах) дээр суурилсан загвар юм. Тэнд өөрийн хувийн санах ойтой байж болох тусдаа процессууд нь самбар (эсвэл агуулах) гэж нэрлэгддэг санах ойн хэсгийг хуваалцдаг бөгөөд үүнийг харилцаанд зориулан ашигладаг. Энэ салбарт хамгийн алдартай загвар бол самбарыг *tuple* зай гэж үздэг Линда хэлний загвар бөгөөд энэ нь процесс дунд хуваалцахад тохиромжтой санах ойн хийсвэрлэл юм. Процессууд нь tuple зайруу/зайгаас илгээх/хүлээн авахад

харилцааны тохиромжтой командуудыг ашиглан бие биентэйгээ харилцдаг. Тиймээс, мессеж солилцох аргыг ашигладаг байсан ч харилцаа нь далд бөгөөд дундын санах ойгоор дамждаг. "координат хэл" гэж нэрлэгддэг контекстоор судлагдсан энэ загвар нь сонирхолтой боловч бусад хоёртой харьцуулахад бага ач холбогдолтой бөгөөд энэ номонд орхисон болно.

Дундын санах ойгоор харилцах

Энэ төрлийн харилцаа нь энэ номын бусад хэсэгт бидний үзсэн зүйлс бөгөөд императив парадигмд бол энгийн юм: Нэг програмын нэг хувьсагч руу хандах эрхтэй хоёр хэсэг нь хувьсагч руу бичиж, утгыг нь унших замаар л харилцах боломжтой. Ерөнхийдөө, дундын санах ойн зэрэгцээ програмчлалын загварт хоёр оролцогч, жишээлбэл, хоёр өөр процесс нь утгыг бичиж, унших боломжтой ижил санах ойн зайд хандах эрхтэй. Энэхүү санах ойн зайг нэгээс олон оролцогчтой хуваалцах боломжтой бөгөөд ингэснээр нэг ижил мэдээллийг нэгээс олон оролцогч хүлээн авдаг өргөн мэдээлэх (broadcast) харилцааг зөвшөөрдөг.

Мессеж солилцоогоор харилцах

Энэхүү харилцааны загварын дагуу оролцогчид санах ойн ямар ч орон зайг хуваалцдаггүй: харилцахын тулд илгээгч нь *send* үйлдлийг ил гүйцэтгэх ёстой бол хосолсон замаар хүлээн авагч нь *receive* үйлдлийг ил гүйцэтгэх ёстой. Түүгээр ч зогсохгүй, логик үүднээс авч үзвэл илгээгч болон хүлээн авагч хоёулаа хамгийн багадаа хоёр оролцогчийг холбосон замыг бий болгохын тулд ихэвчлэн *суваг (channel)* гэж нэрлэгддэг харилцааны зориулалтын бүтцэд хандах чадвартай байх ёстой. Энэ бүтцийн хэрэгжилт нь, түүнчлэн илгээх болон хүлээн авах үйлдлүүд нь хангалттай олон ялгарч болох бөгөөд харилцааны үйлдлийн хийсвэрлэх түвшин болон суурилж буй техник хангамжийн бүтцээс хамаарна. Зарчмын хувьд физик машин болон түүн дээр хэрэгждэг хэл дээрх харилцааны механизмууд өөр өөр байж болно гэдгийг санах нь зүйтэй. Жишээлбэл, мессеж солилцох харилцааг хэрэгжүүлдэг хэл дээр бичигдсэн програм нь сувгуудыг дуурайн илгээх, хүлээн авах үйлдлийг хэрэгжүүлэхийн тулд дундын санах ойг ашигладаг олон процессортой машин дээр ажилладаг. Үүний нэгэн адил зарчмын хувьд тархмал санах ой бүхий олон компьютер дээр дундын санах ойн харилцааг ашигладаг хэлээр бичигдсэн программыг мессеж солилцоход суурилсан техник хангамжийн түвшний харилцаанд суурилуулан ажиллуулж болно.

14.3.2 Синхрончлолын механизм

Одоо байгаа зэрэгцээ байдлын загваруудын хоёрдахь үндсэн тал нь *синхрончлолын* механизмтай холбоотой юм. Эдгээр механизмууд нь янз бүрийн процесс дахь идэвхжилийн харилцан эрэмбийг хянах боломж олгодог бөгөөд тооцооллын зөв байдлыг хангахад зайлшгүй шаардлагатай. Дундын санах ойн харилцааны хүрээнд энэ тал нь тодорхой бөгөөд синхрончлолын механизмууд ил хэлбэрээр бичигддэг. Жишээлбэл, дундын х хувьсагчийг ашиглан харилцаж буй хоёр процессын тохиолдлыг авч үзье: Мэдээжийн хэрэг, хүлээн авах процесс нь х-д зөвхөн үүсгэх процессыг зөв утгыг бичсэний дараа л х-ийн утгыг авах ёстой. Ерөнхийдөө, дундын санах ойтой зэрэгцээ програмчлалын хувьд бид хоёр төрлийн синхрончлолыг авч үздэг: кодын эгзэгтэй тусгай мужуудад хэд хэдэн процесс нэгэн зэрэг нэвтрэх боломжгүй гэдгийг баталгаажуулдаг *харилцан хориглох (mutual exclusion)*, мөн тохирох нөхцөл үүсэх хүртэл процессын гүйцэтгэлийг түр зогсоох боломжийг олгодог *нөхцөлт синхрончлол (condition synchronization)*.

Эдгээр хоёр төрлийн синхрончлолыг *завгүй хүлээлт (busy waiting)*, мөн *spinning* гэж нэрлэдэг) болон *блок* (мөн хуваарьт суурилсан синхрончлол гэж нэрлэдэг) ашиглан гүйцэтгэдэг. Эхний

тохиолдолд хүлээх үйл явц нь нөхцөлийг тасралтгүй шалгадаг циклийг гүйцэтгэх замаар CPU-г идэвхтэй ашигладаг. Хоёр дахь нь хүлээх ёстой процесс нь CPU-г суллаж, бусад хүмүүст ашиглах боломжтой болгодог. Хүлээлтийн нөхцөл батлагдсан үед хуваарилагч нь процессыг дахин идэвхжүүлнэ. Олон төтрлийн механизмууд нь ийм хэлбэрийн синхрончлол, тухайлбал семафор болон монитормыг зөвшөөрдөг. Бид эдгээр бүх техникийг 14.4-р бүлэгт дэлгэрэнгүй харна.

Асинхрон харилцаа

Энэ төрлийн харилцаанд мессеж илгээх, хүлээн авах нь цахим шуудангийн нэгэн адил өөр өөр цаг үед явагддаг. Илгээгч нь `send` үйлдлийг гүйцэтгэх ба дараа нь хүлээн авагч холбогдох `receive` үйлдлээ гүйцэтгэхийг хүлээхгүйгээр тооцооллыг үргэлжлүүлнэ. Энэ тохиолдолд `send` үйлдлийг блоклогддоггүй (`non-blocking`) гэж хэлдэг. Нөгөө талаас `receive` үйлдэл нь блоклогддог: `receive`-г гүйцэтгэсний дараа хүлээн авагчийн процесс нь блоклогдож, илгээгч `send` үйлдлийг гүйцэтгэхийг хүлээнэ - хүлээн авагч нь зөвхөн үүнийг хийсний дараа тооцооллыг үргэлжлүүлж болно. Асинхрон харилцаа нь сувгыг хэрэгжилт дээр зарим хүндрэл гаргадаг, учир нь тэд зарчмын хувьд илгээсэн болон хараахан хүлээн аваагүй дурын тооны мессежийг агуулж болно.

Синхрон харилцаа

Синхрон харилцааны хувьд энэ нь зөвхөн илгээгч болон хүлээн авагч хоёулаа утсаар ярих (хариулагч машин ашиглахгүйгээр) тусгай суваг ашиглан харилцахад бэлэн байх үед л боломжтой. Онолын хувьд синхрон харилцаанд `send` болон `receive` нь нэгэн зэрэг хийгддэг. Практикт үйлдлүүд өөр өөр цаг хугацаанд явагддаг бөгөөд илгээгч нь `send` үйлдлийг гүйцэтгэсний дараа хүлээн авагч харгалзах `receive` үйлдлийг гүйцэтгэсний дараа л тооцооллоо үргэлжлүүлж болно гэсэн үг. Хүлээн авагч нь асинхрон харилцаатай адил байдлаар ажилладаг. Синхрон харилцааны хувьд `send` болон `receive` аль аль нь блоклогддог. Синхрон харилцааны хэрэгжилт нь холболтын сувгийг байгуулахын тулд хязгаарлагдмал хэмжээтэй буфер ашиглаж болно, учир нь өмнөх мессежийг хүлээн авах хүртэл шинэ мессеж илгээх боломжгүй.

Харилцааны механизмуудад хамаардаг дээрх нэр томъёог биелэлтийн механизмтай андуурч буруу ойлгохгүй байхад уншигчдын анхаарлыг хандуулахыг хүсч байна. Хэрэв бид биелэлтийн механизмуудыг авч үзвэл, энэ бүлэгт авч үзэх хэлүүд бүгд асинхрон хэл бөгөөд энэ нь процесс, урсгал, даалгавар эсвэл програм бүр нь бусад процессуудтай шууд холбогдохгүйгээр өөрийн хурдаар (мэдээжийн хэрэг, үндсэн процессороос хамаарна) биелэгддэг.

Үнэн хэрэгтээ янз бүрийн процессын биелэгдэх хугацааг синхрончлох боломжтой синхрон хэл, загварууд байдаг - ашигласан архитектурын төрлөөс эсвэл ашигласан тооцооллын хийсвэр загвар дээр хийсэн таамаглалаас шалтгаална. Эдгээр хэл нь бодит цагийн болон эмбэддэд системд ихээхэн сонирхол татдаг боловч тэдгээрийн хэлэлцүүлэг нь энэ бүлгийн хамрах хүрээнээс гадуур юм.

14.4 Дундын санах ой

Энэ хэсэгт бид дундын санах ойд суурилсан зэрэгцээ програмчлалын загварт ашигладаг синхрончлолын механизмуудыг авч үзэх болно. Өмнө дурьдсанчлан, энэхүү механизм нь хоёр ба түүнээс олон оролцогчдийн хоорондын харилцааг дэмжиж өгдөг бөгөөд эдгээрийг хялбаршуулах үүднээс хувьсагч гэж тодорхойлсон ижил санах ойн хэсэгт хандах боломжийг олгодог. Харилцаа зөв байхын тулд уралдааны нөхцөл болон бусад буруу нөхцлөөс байдлаас зайлсхийхийн тулд бидэнд

тусгай синхрончлолын техник хэрэгтэй. 14.3.1-д дурдсанчлан, дундын санах ойн синхрончлолын хоёр үндсэн хэлбэр нь харилцан хориглох, нөхцөлт синхрончлол юм. *Харилцан хориглох (mutual exclusion)* нь кодын эгзэгтэй муж (*critical section*) гэж нэрлэгддэг дундын нөөцөд ханддаг хэсийг хамтран биелүүлэх боломжийг зэрэгцсэн процессуудад зөвшөөрдөг. Энэ байдлаар биелэлтийн хувьд кодын хэсгийг бүтнээр нь атомик гэж үзэж болно. Нөгөө талаас *нөхцөлт синхрончлол* нь тусгай утга⁴ авдаг хувьсагч зэрэг тохиромжтой нөхцлийг үүсэх хүртэл процессын биелэлтийг түр зогсоох боломжийг олгодог. Эдгээр хоёр төрлийн синхрончлолыг хоёр техник ашиглан хийдэг: завгүй хүлээх (мөн *spinning* гэж нэрлэдэг) болон блок (мөн хуваарьт суурилсан синхрончлол гэж нэрлэдэг).

14.4.1 Завгүй хүлээлт

Завгүй хүлээх үед ээлжээ хүлээх ёстой процесс (эгзэгтэй хэсэгт орох эсвэл өгөгдсөн нөхцөл байдал үүссэн) нь нөхцөлийг үнэн болох хүртэл нь шалгах тасралтгүй циклийг гүйцэтгэдэг. Хүлээлтийн үеэр, CPU идэвхтэй ажилласаар байх тул энэ техникийг өөр өөр процессуудыг синхрончлохын тулд нэг төвлөрсөн процессортой машин дээр ашиглах нь тийм ч уялдаатай биш, учир нь ижил CPU-г ашиглаж буй өөр процессоос гарах нөхцлийг (жишээ нь, буфер хоослох) хүлээж CPU-г сул зогсоох нь ашиггүй. Нөгөөтэйгүүр, энэ нь олон процессортой машин дээр (эсвэл техник хангамжийн түвшинд, жишээлбэл, дата шугам болон дотоод сүлжээн дэх өгөгдөл дамжуулалтыг синхрончлоход) ашигладаг бол үр дүнтэй арга байна. Синхрончлолын олон механизмд идэвхтэй хүлээлтийг ашигладаг: хамгийн чухал нь *түгжээ (lock)*, харилцан хориглолтыг хэрэгжүүлэх) ба саад (*barrier*, нөхцөлт синхрончлолыг хэрэгжүүлэх) юм. Хоёуланг нь харцгаая.

Түгжээ

Нэгдүгээрт, харилцан хориглох механизмыг хэрэгжүүлэхийн тулд бид ямар атомик зааврыг ашиглах боломжтой вэ, өөрөөр хэлбэл тэдгээр нь ажиллаж буй машинд хуваагдахгүй гэдэг нь баталгаатай байхыг тодорхой болгох хэрэгтэй. Ихэвчлэн техник хангамжийн түвшинд Boolean B хувьсагчийн утгыг буцаадаг *test_and_set(B)* атомик заавар байдаг ба тиймээс атомикаар буюу жишээ нь тасалдалгүй байдлаар тухайн хувьсагчид *true* утга олгодог. Ийм зааврыг ашиглан бид харилцан хориглохыг дараах байдлаар хийж чадна.

Эгзэгтэй мужид хандахыг хүссэн процесс бүр дараах бүтэцтэй байна гэж бодъё (ердийнх шигээ бид энд псевдо хэлээ чөлөөтэй ашиглана):

```
process Pi {
    non -critical section;
    acquire_lock( B );
    critical section;
    release_lock( B );
    non -critical section;
}
```

⁴ Харилцан хориглох нь нөхцөлт синхрончлолын онцгой тохиолдол мэт санагдаж болох бөгөөд нөхцөл нь: "Өөр ямар ч процесс эгзэгтэй мужид байхгүй". Гэсэн хэдий ч энэ нь системийн глобал төлөвт хяналт тавих болно, энэ нь нөхцөлт синхрончлолд ерөнхидөө хэрэглэгддэггүй.

Эгзэгтэй муж нь өмнө дурьдсанчлан, бид харилцан хориглох байдлыг баталгаажуулахыг хүсч буй дундын өгөгдлийн бүтцийг багтаасан болно, өөрөөр хэлбэл бид нэгэн зэрэг хандалтаас урьдчилан сэргийлэхийг хүсч байна. Boolean төрлийн дундын B хувьсагчийг false гэж эхлүүлсэн гэж үзье.

```
bool B = false;
```

Энэ хувьсагч нь түгжээ гэж нэрлэгддэг зүйл гэж ойлгох бөгөөд, өөрөөр хэлбэл эгзэгтэй мужид хандахад ашиглах түгжээг юм. Тиймээс test_and_set(B) зааврыг ашиглан бид түгжээ эзээших, суллах функцийг дараах байдлаар тодорхойлж болно.

```
void acquire_lock( ref B: bool ) {  
    while test_and_set( B ) do { skip ; }  
}  
void release_lock( ref B: bool ) {  
    B = false ;  
}
```

(дээрх skip нь ямар ч нөлөө үзүүлэхгүй команд бөгөөд зөвхөн тайлбарыг тодорхой болгох үүднээс жишээнд оруулсан болно). Түгжээ эзэмших процедур нь B дундын хувьсагчийг false утгатай болтол нь шалгадаг давталтаас бүрддэг (мөн энэ техникийг *spin* гэж нэрлэдэг, иймээс энэ техникийг *spin түгжээ* гэж нэрлэдэг) бөгөөд энэ нь эгзэгтэй мужид хандах боломжийг илэрхийлдэг. Тестийг B-д true утга өгөх адилхан атомик үйлдлээр хийх бөгөөд ингэснээр хамтарсан тест хийж үзсэн хэд хэдэн оролдлогын зөвхөн нэг нь (хамтарсан процессуудаар) амжилттай болно. Энэ нь test_and_set-ийн үйлдлийн атомик чадварын ачаар acquire_lock(B)-ыг белүүлж байгаа хэд хэдэн процессуудын зөвхөн нэг нь л эгзэгтэй мужид нэвтэрч чадна гэсэн үг бөгөөд энэ нь харилцан хориглох байдлыг баталгаатай байлгадаг. Энэ механизм нь эгзэгтэй мужруу рүү орохыг оролдож буй олон процессын аль нэг нь амжилттай болдог, өөрөөр хэлбэл үргэлжлэх боломжтой ядаж нэг процесс (эгзэгтэй мужид орж байгаа нэг нь) байдгийг баталгаажуулдаг болохыг анхаарна уу. Өөрөөр хэлбэл, энэ тохиргоог хийснээр бид түгжрэл (deadlock) байхгүй гэдгийг баталгаажуулж чадна. Харамсалтай нь, харилцан хориглох өмнөх хэрэгжилт нь хамтарсан процессортой машинуудын үр ашгийн ноцтой асуудал болж байна. Эгзэгтэй мужид хандахыг оролдож буй процессууд нь while давталт дотор test_and_set үйлдлийг гүйцэтгэх бүртээ дундын хувьсагч рүү бичих үйлдлийг хийсээр байх болно. Хувьсагчийн утгыг шалгахын зэрэгцээ атомик байдлаар гүйцэтгэдэг ийм үйлдлүүд нь санах ой, дата шугам ашиглалтын өрсөлдөөн үүсгэж, улмаар системийн гүйцэтгэлийг бууруулдаг.

Энэ асуудлаас зайлсхийхийн тулд test ба test_and_set техникийг ашиглаж болно. Дундын хувьсагч дээр test_and_set үйлдлийг хийж бичихийн оронд бид цикл бүрт зөвхөн test (үнших) үйлдлийг гүйцэтгэдэг. Тэр үйлдэл амжилттай болсон үед бид test_and_set рүү шилжиж, эгзэгтэй мужид хандах эрх авдаг. Энэхүү техникийн дагуу түгжээ эзэмших функцийг дараах байдлаар гүйцэтгэнэ (суллах нь өмнөх шигээ байна):

```
void acquire_lock( ref B: bool ) {  
    while B do { skip ; }  
    while test_and_set( B ) do {  
        while B do { skip ; }  
    }  
}
```

}

Энэ болон харилцан хориглох өмнөх хэрэгжилт нь нэмэлт асуудал үүсгэдэг. Эрт орой хэзээ нэгэн цагт эгзэгтэй мужид хандахыг хүсч буй процесс амжилтанд хүрэх баталгаа байхгүй. Үнэн хэрэгтээ, эгзэгтэй муж чөлөөлөгдөхөд түүнд хандах өөр процесс үргэлж шууд гарч ирдэг байх тохиолдолд магадлалтай. Учир нь бид процессийн удирдлага, ялангуяа тэдгээрийн төлөвлөгчийн талаар ямар ч таамаглал дэвшүүлдэггүй. Практикт ийм үйлдэл хийх магадлал цөөн мэт боловч эгзэгтэй хэсэг рүү хандахыг хүлээж байгаа эсвэл илүү ерөнхийгөөр бол шилжихийг хүлээж буй процесс эрт орой хэзээ нэгэн цагт амжилтанд хүрэхийг баталгаажуулдаг тусгай алгоритмууд байдаг.

Эдгээр шинж чанаруудыг "шударга байдал (fairness)" гэж нэрлэдэг бөгөөд тэдгээрийн нарийн тодорхойлолт нь бидний судалгааны хамрах хүрээнээс гадуур юм (шударга байдлын тухай өөр өөр ойлголт байдаг бөгөөд зарим тохиолдолд ялгаа нь нарийн байдаг). Энд бид энэ нэр томъёог логикийн хувьд ашиглахаар шийдсэн: Нийтлэг нөөцөд олон өрсөлдөгч байгаа гэж үзвэл, нөөц эзэмших боломжтой нэхэмжлэгч бүр хязгааргүй олон удаа нөөцөд амжилттай нэвтрэх боломжтой байвал нөөцөд хандах удирдлагын бодлого (*сул*) *шударга (fair)* байна. Эгзэгтэй мужид хандахаар олон процесс өрсөлдөж байгаа тохиолдолд тэдгээрийн хооронд шилжилт хийж чадах харилцан хориглох шударга шийдэлд дээрх санааг баримтлан хүрч чадна. Энэ санааг логикийн хувьд энгийн байдлаар хэрэгжүүлдэг алгоритм бол тасалбарын машин юм – боруулалт руу хандах үйлчлүүлэгчийн хандалтыг удирдах олон байрлал. Бид хэрэглэгчдэд өсөх дарааллаар эрэмбэлсэн тоо (тасалбар) түгээгчийг өгч, энэ эрэмбийг даган нөөцөд хандах эрхийг харилцан хориглосон байдлаар олгодог. Энэ алгоритмын хязгаарлалт нь утгууд зарчмын хувьд хязгааргүй өсөх явдал юм. Тусгайлсан бүтээлүүдэд tie-breaker зэрэг хязгааргүй өсөлтөөс зайлсхийх алгоритмуудын жишээг агуулдаг.

Саад

Саад нь ихэвчлэн давталтын аргуудыг (жишээ нь, матрицын манипуляци) ашиглан эцсийн шийдлийг гаргахад тохирсон n өөр процессоор хэрэгжүүлдэг алгоритмтай байдаг параллел програмчлалд ерөнхийдөө ашиглагддаг нөхцөлт синхрончлолын механизм юм. Давталт бүрийн төгсгөлд процесс бүр дараагийн давталт руу шилжихээсээ өмнө бусад бүх процессийн ажиллагаа дуусахыг хүлээх ёстой. Энэ нь кодонд *saad (barrier)* гэж нэрлэгддэг синхрончлолын цэгийг нэвтрүүлснээр бий болох болох бөгөөд процесс бүр цааш үргэлжлэхийн өмнө глобал нөхцөл биелэхийг хүлээж байдаг. $i = 1 \dots n$ -тэй P_i процесс бүр дараах хэлбэрийн загвартай байх болно:

```
process  $P_i$  {  
    while B true do {  
        code of task  $i$ ;  
        wait for all other  $P_j$  to terminate;  
        // barrier  
    }  
}
```

Энэ хүлээлтийг, улмаар саадыг хэрэгжүүлэхийн тулд бид циклийн өмнө тэг дээр эхлүүлсэн, процесс бүр ажлынхаа төгсгөлд нэмэгдүүлэн n утгад хүрэхийг хүлээдэг глобал тоолуурыг ашиглаж болно. Тоолуур руу хандах үйлдлүүд нь атомик хийгдэх ёстой бөгөөд хүлээлт идэвхтэй байж болно.

Саадыг хэрэгжүүлэх олон арга, тэр ч байтугай нэлээд боловсронгуй арга байдаг. Эдгээр нь параллел програмчлалын хувьд нэлээд онцлогтой тул бид энд цаашид авч үзэхгүй.

14.4.2 Төлөвлөгчид суурилсан синхрончлол

Өмнөх хэсэгт үзсэн идэвхтэй хүлээх механизм нь хэд хэдэн сул талтай. Нэгдүгээрт, ихэвчлэн процессоруудаас илүү олон процесс байдаг тул процессууд нь CPU-г ашиглахыг хүлээж байхад нэг процессийг идэвхтэй хүлээхийн төлөө CPU-ийн цагийг ашиглах нь тохиромжгүй байдаг.⁵

Цаашлаад, бидний жишээн дээр болон практикт, синхрончлолд ашигласан хувьсагч нь "энгийн" хувьсагч бөгөөд энэ зориулалтаар тусгай өгөгдлийн төрөл эсвэл зарим хэл шинжлэлийн бүтцэд байхгүй. Өгөгдлийн төрөл, объектын тухай үзсэн агуулгаасаа бид программчлалын алдаа гаргах магадлалыг багасгахыг хүсч байвал анхаарал хандуулах шаардлагатай асуудлуудыг тусад нь байлгахыг хичээх хэрэгтэй, жнь синтаксийн хувьд, эсвэл төрлийн системээр дамжуулах.

Дээрх асуудлыг шийдвэрлэхийн тулд *блок дээр суурилсан* механизм гэж нэрлэдэг төлөвлөгчид суурилсан (scheduler-based) синхрончлолын механизмыг танилцуусан. Төлөвлөгч дээр суурилсан синхрончлолд бид процессыг хүлээх үед процессын хүлээж буй эвентийг тухайн процессыг тодорхойлсон мэдээлэлтэй холбодог. Эвент тохиолдоход, төлөвлөгч тухайн эвенттэй холбогдсон процессууд биелэгдэхэд бэлэн байгааг мэддэг. Эдгээр синхрончлолын механизмууд нь ихэвчлэн тусгай утга авч болох хувьсагчийг хэрэглэдэг бөгөөд эдгээр утгууд дээр синхрончлолд зориулсан тусгай үйлдлүүдийг дэмждэг. Тиймээс бид тэдгээрийг дор хаяж ойлголтын түвшинд өгөгдлийн төрөл гэж харж болно, гэхдээ тэдгээр нь дээд түвшний хэл дээр бүтэц хэлбэрээр ховорхон харагддаг.

Дараах хэсэгт бид энэ төрлийн хоёр үндсэн механизмыг авч үзэх болно, тухайлбал семафор болон монитор нь харилцан хориглох болон нөхцөлт синхрончлолыг дэмждэг.

Семафор

1960-аад онд Дийкстрагийн (Dijkstra) танилцуулсан *семафор* дээр суурилсан механизм нь дундын санах ойн системд зориулсан анхны ил синхрончлолын хэрэгсэл байсан бөгөөд өнөөг хүртэл үндсэн үйлдлийн системийн түвшинд хэрэгжсэн хамгийн чухал синхрончлолын нэг хэвээр байна. ALGOL 68-д анх танилцуулагдсан семафорыг бараг бүх сан зэрэгцээ програмчлалд ашигладаг бөгөөд SR, Modula-3 зэрэг зарим хэл дээр гарч байдаг.

Нэрнээс нь харахад бид семафорыг "мөргөлдөөнөөс" зайлсхийхийн тулд процессийн "трафикиг" зохицуулдаг бүтэц гэж харж болно. Энэ нь төмөр замын хөдөлгөөнийг зохицуулдаг бодит төхөөрөмжтэй төстэй бөгөөд зориулалтын семафорын өнгө нь замын хэсэг чөлөөтэй эсэхийг илтгэдэг тул галт тэрэг нэвтрэхийг зөвшөөрдөг эсвэл хориглодог. Галт тэрэг замын хэсгийг эзлэх (ногооноос улаан) эсвэл суллах (улаанаас ногоон) үед семафорын өнгийг тусгай үйлдлээр удирддаг.

6

⁵ Идэвхтэй хүлээх хугацаа нь өөр процессыг гүйцэтгэхэд шаардлагдах контекст шилжилтийн хугацаанаас бага байх үед энэ зарчмын онцгой тохиолдол байдаг.

⁶ Энд бид хялбаршуулан галт тэрэгний хоёртын хувилбарыг замын хэсэгт байгаа эсэх талаар авч үзэж, өөр галт тэрэг энэ хэсгээс хөдөлж/ орж ирж байгааг илтгэх гуравдахь өнгөөс (ихэвчлэн шар) зайлсхийж байна.

Метафор нь программын хэсгүүдэд процессууд хандах эсэхийг тогтоох утгуудыг (өнгөнүүдэд харгалзах) тодорхойлдог өгөгдлийн төрөл байх семафор болж хувирдаг. Бидний sem гэж нэрлэх семафор өгөгдлийн төрөл нь:

- ≥ 0 байх бүхэл тооноос бүрдэх утгын олонлог;
- P ба V гэж нэрлэгддэг хоёр атомик үйлдэл.

Семафор төрлийн s хувьсагчийг ердийн байдлаар зарласан:

sem s

процесс хоорондын s-ийн утгад суурилсан синхрончлолыг дэмждэг дундын бүтцийг өгдөг бөгөөд P ба V хоёр үйлдлээр, P нь семафор руу хандах боломжийг олгодог, V нь семафоруыг чөлөөлдөг.

Илүү нарийвчилбал, хэрэв P процесс нь

P(s)

үйлдлийг гүйцэтгэдэг бол үр дүн нь дараах байдалтай байна. Хэрэв s-ийн утга > 0 бол s-ийг багасгаж, үйлдэл дуусах тул процесс нь P(s)-ийг дагах кодын биелэлтээр (өөрөөр хэлбэл, чухал хэсэгт хандах) үргэлжлүүлнэ. Эсвэл хэрэв s хувьсагч 0 утгатай бол үйлдэл өөрөө (мөн P процессийг өргөтгөх) s утгыг > 0 авах хүртэл блоклодог. Энэ блоклох үйлдлийг ихэвчлэн үйлдлийн системийн цөмийн түвшинд гүйцэтгэдэг. Энэ нь бусдынхаа хэрэглэж болох CPU-г чөлөөтэй байлгаж буй P процессийг s хувьсагч > 0 утгатай болох эвентийн дохиогоор үйлдлийн системээс "сэрээнэ" гэсэн үг юм. Дээр дурьдсанчлан, P(s)-ийг ашигласан эдгээр үйлдлүүдийг нэг атомик үйлдлээр гүйцэтгэдэг гэж үзэх ёстой. Энэ нь жишээлбэл, хэрэв s-ийн утга 1 бөгөөд хоёр процесс P(s) үйлдлийг гүйцэтгэж байгаа бол зөвхөн нэг үйлдэл нь эгзэгтэй мужид орж, нөгөө нь блоклогддог гэсэн үг юм.

үйлдэл

V(s)

нь P(s)-д даатгасан нөөц одоо чөлөөтэй болсон гэсэн дохиог өгөх боломжийг олгодог. Үүний тулд V(s) нэг атомик үйлдлээр s-ийн утгыг нэмэгдүүлнэ. V(s)-ийг биелүүлдэг процесс хэзээ ч блоклогдохгүй.

s хувьсагч нь зөвхөн 0 ба 1 утгыг авах боломжтой бол бид хоёртын семафорын тухай ярьж байна. Хэрэв s нь сөрөг бүс утгыг авч чаддаг бол бид нийтлэг семафортой болно.

Блоклогдсон P үйлдлүүдийг "сэрээх"-ийг ихэвчлэн процессын хуваарь гаргагч зохицуулдаг. Иймд эдгээр үйлдлийг шударга явуулах, эгзэгтэй мужуудад процессийн хандах шударга байдал нь төлөвлөгчийн өөрийнх нь шударга байдлын таамаглалаас хамаарна. Ер нь семафорын хэрэгжилт нь P үйлдлийг дуусгахыг хүлээж буй блоклогдсон процессуудын дарааллыг First-In-First-Out (FIFO) горимоор удирддаг: хамгийн түрүүнд блоклогдсон процессууд эхлээд сэрдэг. Тиймээс, хэрэв хангалттай V үйлдэл байгаа бол шударга ёсыг баталгаажина.

Семафор ашиглах жишээ болгон Дийкстрагаас үүдэлтэй сонгодог синхрончлолын асуудал болох dining philosophers асуудлыг авч үзье. 14.1-р зурагт үзүүлсэн шиг таван философич дугуй ширээний ард сууж байна. Философич бүр хооллох, бясалгах үйлдлүүдийг ээлжлэн хийдэг. Философич бүр зөвхөн тавагнахаа хажуу талд байрлуулсан сэрээг ашиглаж чаддаг бөгөөд

философич бүрт идэхийн тулд хоёр сэрээ хэрэгтэй. Философичдын тоогоор сэрээ байдаг тул философич бүр нэг сэрээ авбал (жишээ нь, тавагны баруун талд байгаа) хэн ч идэж чадахгүй. Энэ бол түгжрэл (deadlock) нөхцөл байдлын жишээ юм. Процесс бүр (философич) өөр процессоор нөөцийг (сэрээ) чөлөөлөгдөхийг хүлээх ёстой боловч чөлөөлөх хамаарлын хэлхээ нь цикл учраас хэн ч үргэлжлүүлж чадахгүй. Тиймээс, энэ хувилбарт түгжрэлээс зайлсхийхийн тулд бид тойрон хүлээхээс (эхний процесс нь гурвыг хүлээж буй хоёрыг гэх маягаар эргээд эхнийхийг хүлээнэ) зайлсхийх хэрэгтэй. Жишээлбэл, эхний философич эхлээд баруун сэрээ, дараа нь зүүн сэрээ, бусад нь эхлээд зүүн сэрээ, дараа нь баруун сэрээ авдаг гэж үзвэл ийм байдал хүрнэ. Иймээс уг синхрончлолын асуудлыг шийдэх арга нь дараах байдалтай байна. Бидэнд 5 семафорын forks массив байгаа, сэрээ тус бүрд нэг, бид бүгд үүнийг 1 гэж эхлүүлнэ гэж бодъё.

Бид мөн philosopher[i]-ээр нийтлэг i-р философчийг тэмдэглэдэг (массивын индексүүдтэй адилаар $i \in [0,4]$ байна: сэрээний тоо i нь философич i-ийн баруун талд байгаа). Сэрээ сонгох нь P үйлдлээр хийгддэг бол чөлөөлөх нь V үйлдлээр явагддаг.

```
sem[] forks[5];
for ( i = 0, i < 5, i += 1 ) { forks[ i ] = 1; }

process Philosopher( i ){
    sem first_fork, second_fork;
    if( i == 0 ){
        first_fork = forks[ 4 ];    // the one on the right
        second_fork = forks[ 0 ];    // the one on the left
    } else {
        first_fork = forks[ i ];     // the one on the left
        second_fork = forks[ i-1 ]; // the one on the right
    }
    while true {
        think;
        P( first_fork ); // acquire first
        P( second_fork ); // acquire second
        eat;
        V( first_fork ); // release first
        V( second_fork ); // release second
    }
}
```

```
Philosopher(0) || Philosopher(1) || Philosopher(2) || Philosopher(3) || Philosopher(4)
```

Кодын сүүлийн мөр нь таван философичтой харгалзах процессүүдийг зэрэгцээ биелэхийг харуулж байна. Шийдлийн зөвийг батлах нь өчүүхэн зүйл юм. Үнэхээр, боломжит биелэлтийг нэгийг дэлгэе. Бид эхний сэрээгээр (зүүн талд байгаа) forks[1]-ийг авдаг Philosopher(1)-ээс эхэлж, дараа нь (мөн зүүн талд нь) forks[2] сэрээг авдаг Philosopher(2) гэх мэт forks[4] (түүний зүүн талд) сэрээг Philosopher(4) авна. Одоо Philosopher (0) эхний сэрээгээ авах ээлж ирлээ. Гэсэн хэдий ч, Philosopher (0) циклийг гүйцээхийн оронд (зүүн-эхний авалтууд гацаанд орно) баруун талын сэрээ авахыг оролддог. Энэ бол Philosopher (4) аль хэдийн олж авсан сэрээ[4] юм. Philosopher (0) хийж чадах цорын ганц зүйл бол бусад философичдод хоёр дахь (баруун) сэрээгээ сонгох боломжийг олгон, тэр сэрээг бэлэн болтол хүлээх явдал юм (Philosopher (1) нь forks [0]-ийг, Philosopher (2) нь forks [1]-ийг

гэх мэт Philosopher (4) хүртэл авна). Philosopher (4) болон Philosopher (1) сэрээгээ суллахад Philosopher (0) эцэст нь сэрээгээ (эхлээд баруун, хоёрдугаарт зүүн) авч, зохих хоолоо идэх боломжтой болно. Бид 14.6.1-д || операторын талаар илүү дэлгэрэнгүй авч үзсэн.

Монитор

Өмнөх бүлгүүдэд бид хийсвэрлэх механизм нь өгөгдлийн бүтцийн хүрээнд хэрхэн чухал үүрэг гүйцэтгэдэг болохыг харсан. Тухайлбал, бид эхлээд хийсвэр өгөгдлийн төрлүүд, дараа нь объектуудын үндсэн санааг нарийвчлан шинжилж, датумын хийсвэр дүрслэл болон түүн дээр хийх боломжтой үйлдлүүдийг инкапсуляц хийсэн бүтцийг бий болгох нь яагаад чухал болохыг ойлгохыг хичээсэн. Энэ үүднээс авч үзвэл семафорууд нь хангалттай бүтэц, хийсвэрлэх механизмыг санал болгодоггүй доод түвшний бүтэц учраас дутагдалтай байдаг. Үнэн хэрэгтээ семафор нь үндсэндээ хэд хэдэн процесс дундын хувьсагч бөгөөд P ба V-г ашиглан энэ хувьсагчийг зөв ашиглах, улмаар зөв байх бүх эрсдэлийг хангах нь програмистын үүрэг юм (буруу семафор ашиглах, P эсвэл V-г орхих, P ба V-ээс өөр үйлдэл ашиглах гэх мэт). Семафорууд нь харилцан хориглох болон нөхцөлт синхрончлолын хэрэгжүүлэлтэд хоёуланд нь ажиллах боломжтой гэдгийг анхаарна уу. Энэ нь бүрэн хангалттай бус, учир нь хоёр ойлголт өөр бөгөөд хөгжүүлэгчдэд программыг ойлгоход нь туслахын тулд энэ ялгаатай байдал нь хэл шинжлэлийн түвшинд гарч ирвэл сайн байх болно.

Эдгээр дутагдлыг арилгахын тулд 1970-аад онд Дийкстрагийн санал болгосон мониторуыг нэвтрүүлж, улмаар Бринч Хансен, К.А.Р. Ноаре хөгжүүлэн, сүүлд нь Java зэрэг олон програмчлалын хэлэнд ашигласан. Монитор нь хийсвэр объектын дүрслэлийг төлөвтэй хамт инкапсуляц хийсэн хийсвэр өгөгдлийн төрөл бөгөөд тухайн объект дээр ажиллах зөвшөөрөгдсөн үйлдлүүдтэй. Илүү хэлвэл, монитор нь объектын төлөвийг илэрхийлэх байнгын хувьсагчдыг, эдгээр хувьсагч дээр ажиллах процедурууд болон мониторуыг үүсгэх үед ашигладаг эхлүүлэх командуудыг агуулдаг.

Бид ямар нэгэн тодорхой хэлээр ярихгүйгээр мониторуын зарлалтыг дараах бүтэцтэй өгөгдлийн төрөл болгон дүрсэлж болно:

```
monitor MonitorName {  
    variables;  
    initialisation commands;  
    procedures;  
}
```

Монитор байгаа бол байнгын хувьсагч байдаг (мэдээжийн хэрэг, процедур нь ашиглалтын хугацаа багатай өөрийн гэсэн локал хувьсагчтай байж болно). Тэдгээрийн дотор нөхцөлт хувьсагч гэж нэрлэгддэг тусгай хувьсагч байдгаас бид нөхцөлт синхрончлолыг зөвшөөрдгийг харж болно. Мониторуын гадна талд зөвхөн процедурын нэрс (тэдний signature-тай) харагдах бөгөөд энэ нь байнгын хувьсагч дээр ажиллахад объектын private харагдах байдлаар тодорхойлсон талбаруудтай адил (гадаад) процессууд ашиглаж болох зөвшөөрөгдсөн үйлдлүүд юм. Байнгын хувьсагчдыг монитор үүсгэх үед болон аливаа процедурыг ашиглахаас өмнө холбогдох командуудаар эхлүүлдэг.

Монитор ашигласан зэрэгцээ программын хувьд бид идэвхгүй болон идэвхтэй бүрдэл хэсгүүдийг ялгаж чаддаг. Эхнийх нь дээр дурдсантай төстэй зарлалтын дагуу мониторууд бөгөөд өөр өөр процессууд дундын өгөгдлийн бүтцийг агуулдаг. Сүүлийнх нь процессууд бөгөөд эдгээр нь мониторуудын процедурыг (мөн шууд бусаар хувьсагчдыг) ашиглан харилцан ажилладаг.

Энэхүү эхний тайлбараас харахад мониторын давуу тал нь семафоруудаас илт харагдаж байна: Монитор ашиглаж буй программист нь харьцангуй бие даасан байдлаар процессыг хөгжүүлэхдээ мониторын гадаад интерфэйсийг зөвхөн мэддэг бөгөөд түүний процедур хэрхэн хэрэгжиж байгаагаас, зарим талаараа эдгээр процедурыг бусад процессууд хэрхэн ашиглаж байгааг үл анзааран орхидог. Энэ нь хийсвэр өгөгдлийн төрөл ба объектуудын ердийн давуу тал боловч энд илүү хамааралтай гэж үздэг: дараалсан програмчлалаас ялгаатай нь мониторыг өөр өөр цаг хугацаа, хурдтайгаар нэгэн зэрэг идэвхтэй ажилладаг төрөл бүрийн зэрэгцээ процессууд дундаа хэрэглэдэг. Иймд программыг ойлгох үүднээс, юуны түрүүнд түүний зөв байдлыг хангах үүднээс процесс бүрийг харьцангуй бие даасан байдлаар хөгжүүлэх чадвартай байх нь чухал юм. Мэдээжийн хэрэг, харилцан хориглох болон синхрончлох асуудлыг шийдэх ёстой, гэхдээ монитор ашиглах нь эдгээр асуудалд тусалдаг.

Үнэн хэрэгтээ, мониторуудыг ашиглан харилцан хориглох нь программист юу ч хийх шаардлагагүйгээр өөрөө бүтээгдсэнээр далд баталгаатай байдаг: мониторын процедур нь *тодорхойлолтоор* харилцан хориглох байдлаар хийгддэг. Илүү нарийвчлалтайгаар бид процессоор дуудагдсаны дараа биелэгддэг *идэвхтэй* хяналтын процедурыг тодорхойлдог. Мониторын зааварчилгаа нь өөр өөр процедурын тохиолдол эсвэл ижил процедурын аль ч үед мониторын процедурын хоёр идэвхтэй тохиолдол байж болохгүй гэдгийг баталгаажуулдаг. Мониторыг хэрэгжүүлэх нь мэдээжийн хэрэг доод түвшний зохих механизмүүдыг (түгжээ, семафор эсвэл бүр тасалдал дарангуйлал) ашиглан энэхүү зааварчилгааг дагадаг байхыг баталгаажуулах ёстой.

Нөгөө талаас, монитор ашиглаж байсан ч программист мониторын нөхцөлт хувьсагчдыг ашиглан нөхцөлт синхрончлолыг ил зааж өгөх ёстой бөгөөд энэ нь тусгай нөхцөл үүсэхийг хүлээх үед процессын биелэлтийг түр зогсоох боломжийг олгодог. Гэхдээ, энэ тохиолдолд ч гэсэн хувьсагчдад зөвхөн монитор дотор тодорхойлсон процедурын тусламжтайгаар хандах боломжтой бөгөөд зөвхөн тусгай үйлдлүүдийг ашиглана. Илүү дэлгэрэнгүй авч үзвэл нөхцөлт хувьсагчийг монитор дотор зохих синтаксаар зарлаж болно, жишээлбэл,

cond varname

Бид ийм хувьсагчийн утгыг блоклогдсон процессуудын дараалал гэж онолын хувьд үзэж болно. Ийм хувьсагч дээр байж болох хамгийн чухал хяналтын үйлдлүүд нь дараах байдалтай байна.

empty (varname)

varname-ийн утгыг илэрхийлэх дараалал хоосон байвал үнэн, үгүй бол худал буцаана.

wait (varname)

Энэ үйлдлийг гүйцэтгэх процессыг хааж, varname хувьсагчтай холбоотой дарааллын төгсгөлд байрлуулна. Үүнтэй нэгэн зэрэг процесс нь мониторын хэрэглээг чөлөөлснөөр (бусад) процессуудад ашиглаж болно.

Эцэст нь бид процессыг сэрээх боломжийг олгодог үйл ажиллагаатай болсон. Энэ бол

signal (varname)

хэрэв varname дараалал хоосон биш бол эхэнд байгаа процессыг сэрээх, өөрөөр ямар ч нөлөө үзүүлэхгүй. Ийм байдлаар өмнө нь wait-ээр блоклогдсон процессыг FIFO дарааллын удирдлагын бодлогын дагуу signal-аар дахин биелүүлнэ: хаагдсан эхний процессийг мөн хамгийн түрүүнд сэрээдэг. Анхааралтай уншигч энэ үед асуудал байгааг анзаарсан байх болно: wait ба signal нь зөвхөн монитор дотор тодорхойлогдсон (гаднаас нь дуудагдах боломжгүй) процедуруудыг ашиглах боломжтой үйлдлүүд тул signal-г гүйцэтгэж буй процесс болон өмнө нь wait-ийг гүйцэтгэсэн сэргээгдсэн процесс хоёулаа "монитор дотор" байрладаг. Өмнө нь харсанчлан энэ нь боломжгүй, учир нь ямар ч үед зөвхөн нэг мониторийн процесс ажиллах боломжтой. Ерөнхийдөө энэ асуудлыг шийдэх хоёр арга байдаг:

Дохио өгөх болон Үргэлжлүүлэх signal илгээсэн процесс нь биелэлтийг үргэлжлүүлэх бол нөгөө нь зөвхөн "virtually" сэрдэг, өөрөөр хэлбэл, монитор руу орох үед түүнийг дахин гүйцэтгэх боломжтой гэдгийг харуулахын тулд төлөвийг нь өөрчилдөг.

Дохио өгөх болон Хүлээх Энэ тохиолдолд сэрээж буй процесс шууд ажиллаж, монитор руу хандах боломжтой болно. Процесс нь блоклогдохын оронд signal илгээдэг (энэ тохиолдолд бид үүнийг дарааллын толгой эсвэл сүүл хэсэгт байрлуулж болно).

14.5 Мессеж солилцоо

Өмнө дурьдсанчлан, хуваарилагдсан архитектурын хувьд хамтарсан компьютер эсвэл сүлжээ байхаас үл хамааран хамгийн түгээмэл хэрэглэгддэг харилцааны загвар нь мессеж солилцоход суурилдаг. Энд бид энэ загварыг нарийвчлан шинжилж, түүнийг тодорхойлдог хамгийн чухал талууд, тухайлбал, оролцогдыг тодорхойлох механизмууд (эсвэл нэрлэх механизмууд), асинхрон ба синхрон илгээх, хүлээн авах механизмууд дээр анхаарлаа хандуулна.

14.5.1 Нэрлэх механизм

Харилцааны хоёр оролцогч бие биетэйгээ "ярилцаж" чадахын тулд хэнд мессеж илгээх, хэнээс мессеж хүлээн авахаа мэддэг механизмтай байх ёстой. Өөрөөр хэлбэл, бид оролцогчдийг өөрсдөө тодорхойлох, эсвэл нэрлэх механизм хэрэгтэй байна. Энэ хэсэгт бид гурван хувилбарыг авч үзэх болно. Дараах зүйлд бид харилцааны хоёр хэсгийг *илгээгч* ба *хүлээн авагч* гэж нэрлээд эдгээрийг процессууд гэж үздэг (ердийнх шиг, тэдгээр нь урсгал эсвэл өөр зүйл байж болно).

Нэр

Бидний судалж буй эхний механизм нь магадгүй хамгийн ойлгомжтой юм: процессууд нь давтдагдахгүй танигчаас бүтэх нэртэй бөгөөд бие биетэйгээ харилцахдаа энэ нэрийг илгээх, хүлээн авах үйлдлүүдийн аргумент болгон ашигладаг. Харилцааны энэ аргыг Hoare-ийн CSP (Communicating Sequential Processes) хэл дээр нэвтрүүлсэн бөгөөд дараа нь PVM (Parallel Virtual Machine) болон MPI сан зэрэг бусад тохиолдолд хэрэглэсэн. Тиймээс, жишээлбэл, бид дараах байдлаар тодорхойлсон A ба B хоёр процесстэй байж болно

```

process A {
  ...
  B!v;
  ...
}
process B {
  ...
  A?v;
  ...
}

```

Энд А процесс нь v утгыг В процесс руу илгээдэг бөгөөд энэ нь үүнийг у хувьсагчаар хүлээн авдаг (CSP тэмдэглэгээнд! болон ? нь илгээх, хүлээн авах үйлдлийг тус тус заадаг).

Порт

Харилцааны оролцогчдыг тодорхойлох хоёр дахь арга нь портуудыг ашигладаг. Онолын хувьд оролтын порт нь хүлээн авах процесст нэвтрэх цэгийг хангадаг төхөөрөмж юм. Үүний нэгэн адил гаралтын порт нь илгээгчийн процесст гарах цэгийг хангадаг. Портуудыг ашиглах гол тохиолдол нь тэдгээр нь энгийн байдлаар ижил физик холбооны төхөөрөмжийг олон процессын хооронд хуваалцах боломжийг олгодог. Жишээлбэл, TCP гэх мэт дамжуулалтын түвшний холбооны протоколыг авч үзье. Хэрэв А процесс нь IP хаяг нь *foo* (мэдээжийн хэрэг бид бэлгэдлийн нэрийг ашиглав) гэсэн машин дээр байгаа процесстой TCP холболт үүсгэхийг хүсвэл А нь тухайн хаяг болон холбогдохыг хүссэн портын дугаар х (0-ээс 65535 хооронд) хоёуланг нь зааж өгөх ёстой. Энэ нь ижил физик холбооны төхөөрөмжийг (жишээ нь, ижил Ethernet порт) ашигладаг *foo* хаяг дээр нэгэн зэрэг идэвхтэй (машин нь хамтарсан програмчлалыг зөвшөөрдөг гэж үзье) олон өөр процессуудыг машин дээр ажиллуулах боломжтой болгодог. А-тай харилцахыг хүссэн "right" процесс нь х портыг сонсох процесс байх болно.

Ада бол *entry-calls* гэж нэрлэгддэг портуудыг ашигласан анхны хэлнүүдийн нэг юм. Даалгавар (Ада процесстэй эквивалент) нь үнэндээ оролтын портыг дараах байдлаар зааж өгч болно:

```

task TaskType is
  entry inputPort( data: in integer );
  entry outputPort( data: out integer );
end TaskType;

```

Энэ нь TaskType төрлийн даалгавар нь inputPort болон outputPort нэртэй хоёр порттой болохыг заадаг. Хэрэв даалгаврын их биеийг нь дараах байдлаар тодорхойлогдвол

```

task body TaskType is
  ...
  accept inputPort( data: in integer ) do .... end inputPort
  ....
  accept outputPort( data: out integer ) do ... end outputPort
  ...
end TaskType;

```

дараа нь хүлээн авах тушаал нь TaskType төрлийн Pr процесс нь inputPort портоос бүхэл тоон төрлийн data хувьсагч агуулсан мессежийг хүлээн авч болохыг заана. Энэ утгыг хүлээн авсны дараа do командыг дараах програмын хэсэг ажиллана. Энэ нь outputPort-той төстэй бөгөөд ялгаа нь энэ

тохиолдолд out параметр нь дуудагчаас (жишээ нь, мессеж хүлээн авагч)-аас дуудлага хийгч (илгээгч) рүү дамжуулах горимыг зааж өгдөг.

Pr процесс руу мессеж илгээхийн тулд илгээгчийн процесс зүгээр л дараах хэлбэрийн командыг биелүүлнэ.

```
Pr .inputPort( D );
```

Энд D нь бидний дамжуулахыг хүссэн утгыг агуулсан хувьсагч юм. Тухайн процесс бүр процессын төрлийн зарлалт болон процессын нэрээр тодорхойлогдсон өөрийн портуудтай болохыг анхаарна уу. Тиймээс дээр дурдсанчлан илгээсэн мессежүүд зөвхөн нэг процесст хүрдэг.

Суваг (Channel)

Харилцааг бий болгох хамгийн сүүлийн арга бол *суваг* ашигладаг. Эдгээр нь анх Оссам-д програмчлалын хэлний түвшинд гарч ирсэн бөгөөд дараа нь янз бүрийн онолын загварууд (жишээ нь, λ -calculus) болон хэлүүдэд өргөн хэрэглэгддэг. Суваг нь үзэл баримтлалын хувьд хоёр оролцогчийг холбох боломжийг олгодог аливаа харилцааны бүтцийн хийсвэрлэл юм. Энэ утгаараа дундын хувьсагч эсвэл мониторыг хүртэл сувгийн хэрэгжилт гэж үзэж болно. Гэхдээ ихэвчлэн суваг гэдэг нэр томъёо нь тусдаа төхөөрөмжүүдийг холбосон физик сувгууд байдаг бодит холбоо сүлжээний хийсвэр загваруудад зориулагдсан байдаг. Дараах зүйлд бид сувгууд хэрхэн хэрэгжиж байгаа талаар ямар нэг таамаглал дэвшүүлээгүй ч бид энэ нэр томъёоны хийсвэр хэрэглээгээр хязгаарлагдана.

Бага зэрэг нарийвчлалтай байхыг хичээсэн ч ердийнх шигээ ямар нэг хэлийг ашиглахгүйгээр кодыг харж болно.

channel chanName(Type)

chanName танигч нь сувгийн нэр болон тухайн сувгаар илгээсэн мессежийн Type-г илэрхийлсэн сувгийн зарлалт юм. Мэдээжийн хэрэг, нэг талбарт хязгаарлалт нь энгийн байх үүднээс хийгдсэн: бидэнд тохирох нэрээр тодорхойлогдсон өөр өөр төрлийн талбарууд байж болно. Илгээгч болон хүлээн авагч хоёулаа харилцахын тулд сувгийн нэрийг мэддэг байх ёстой. Өмнөх зарлалт дээр дурьдсанаар харилцаа нь chanName суваг руу дараах үйлдлээр өгөгдөл илгээх замаар хийгддэг.

send chanName(data)

илгээгчээр гүйцэтгэгдэх, мөн сувгаас өгөгдөл хүлээн авахдаа

receive chanName(var)

хүлээн авагчаар гүйцэтгэгдэнэ. Type төрлийн data, var хувьсагчид: эхнийх нь дамжуулах утгуудыг агуулдаг бол хоёр дахь нь харилцаа дууссаны дараа хүлээн авсан утгуудыг агуулна. Практикт хэрэглээнд байгаа харилцааны янз бүрийн механизмд тохирсон өөр өөр сувгийн загварууд байдаг. Тодруулбал, өгөгдөл нь зөвхөн нэг чиглэлд урсах боломжийг олгодог *one-way сувгийг* хоёр чиглэлд дамжуулдаг *two-way сувгаас* ялгадаг.

Мөн нэр томъёоны хувьд илгээгчийг хүлээн авагчтай холбох нэг талын сувгийг *линк* гэж нэрлэдэг бол заримдаа хэд хэдэн илгээгчийг нэг хүлээн авагчтай холбодог (нэг талын) сувгийг *оролтын порт* гэж нэрлэдэг болохыг анхаарна уу. Эцэст нь бид олон илгээгч олон хүлээн авагч руу

илгээж болох олон-олон харилцаатай үед сүвгийн *шуудангийн хайрцгийг (mailbox)* гэж нэрлэдэг. Доор, өөрөөр заагаагүй бол сүвгүүд нь ийм төрлийн байна гэж бид үзэж байна.

Семнатикийн үүднээс бид сүвгийг илгээсэн, хараахан хүлээж аваагүй мессежүүдийн дараалал гэж харж болно. Ийм дарааллын үрт нь харилцааны асинхрон эсвэл синхрон эсэхээс хамаарна гэдгийг бид дараагийн хоёр хэсэгт үзэх болно.

14.5.2 Асинхрон харилцаа

Энэ болон дараагийн хэсэгт бид мессеж солилцох замаар харилцах үндсэн хоёр аргыг задлан шинжилнэ. Бид үүнийг сүвгүүдыг ашигладаг харилцааны бүтцэд хамааруулан хийдэг боловч процессын ил нэр эсвэл портуудыг ашиглах үед дараах зүйлсийг анхаарч үзэх хэрэгтэй.

Асинхрон харилцаанд илгээгч нь мессеж илгээсний дараа хүлээн авагчийг хүлээн авах үйлдлийг гүйцэтгэхийг хүлээхгүйгээр тооцоогоо үргэлжлүүлнэ. Өөрөөр хэлбэл, асинхрон харилцаагаар дараах үйлдлийг блоклогдохгүй.

send chanName(data)

Энэ үйлдлийн үр дүнд chanName сувагтай холбоотой мессеж жагсаалтад data хувьсагчийн утгыг агуулсан мессежийг нэмэх бөгөөд үүний дараа send үйлдлийг гүйцэтгэсэн процесс тооцооллоо үргэлжлүүлж болно.

Энд бид суваг руу хандах үйлдэл нь атомик шинж чанартай, мөн мессеж илгээх ажиллагаа найдвартай (өөрөөр хэлбэл илгээсэн мессежийг үнэхээр, алдаагүй хүлээн авдаг) гэж таамаглаж байна. Бид мөн сүвгийн дарааллыг FIFO гэж үздэг тул илгээсэн эхний мессеж нь эхний хүлээн авах мессеж болно. Мэдээжийн хэрэг, эдгээр таамаглалын үнэн зөв нь сүвгүүдыг хэрэгжүүлж буй үндсэн бүтцээс хамаарна. Наад зах нь зарчмын хувьд асинхрон холболттой бол сувагтай холбоотой дараалал нь хязгааргүй үрттай байдаг, учир нь аливаа хүлээн авалтын өмнө дур мэдэн олон тооны мессеж илгээгдэж болно.

Өмнө дурьдсанчлан хүлээн авалт нь дараах үйлдлийг ашиглан ажилладаг.

receive chanName(var)

Энэ үйлдлийн үр дүн дараах байдалтай байна. Хэрэв chanName сувагтай холбоотой дараалал хоосон биш бол receive үйлдэл нь дарааллаас эхний мессежийг авах ба устгана. Тухайн мессежийн өгөгдлийг var хувьсагчид оноодог (мөн энэ тохиолдолд бид дараалалд хандах хандалтыг атомик үйлдлээр хангадаг гэж үздэг). Хэрэв мессежийн дараалал хоосон байвал дараалалд дор хаяж нэг мессеж болох хүртэл receive үйлдлийг блоклож, түүнийг биелүүлсэн процессыг блоклосон байна. Энэ эвент тохиолдоход блоклогдсон процесс сэрдэг - бид блок дээр суурилсан синхрончлолыг авч үзэх үед өмнө нь тайлбарласан аргуудын дагуу. receive үйлдэл нь үргэлж блоклодог тул зарим тохиолдолд бид хоосон суваг дээр receive хийхгүй (мөн блоклох) зайлсхийхийн тулд суваг хоосон эсэхийг шалгах боломжийг олгохыг хүсдэг. Үүний нэгэн адил бид процессуудад receive үйлдлүүдийн завсарлагыг тодорхойлохыг зөвшөөрч болох бөгөөд энэ нь дуудалтаас хойш тодорхой хугацааны дараа хоосон суваг дээрх хүлээлтийг тасалж болно. Зарим хэлүүд эдгээр тохиолдлуудыг шийдвэрлэх тусгай бүтэцтэй байдаг. Цаашилбал, зарим тохиолдолд мессеж хүлээн авах үйлдэл нь харилцааны командыг ил ашигладаггүй харин далд хэлбэрээр хийгддэг. Энэ хэргийг 14.5.4-т алсын процедур дуудалтыг (Remote Procedure Calls) хэлэлцэх үед авч үзэх болно.

Асинхрон харилцах хэрэглээний жишээ болгон бид хоёр үйлчлүүлэгч болон серверийн хоорондын энгийн харилцан үйлчлэлийг харж байна. Үйлчлүүлэгч бүр сервер рүү үйлчлүүлэгчийн нэр (1 эсвэл 2) болон боловсруулагдах өгөгдлийг (char төрлийн) зааж өгөх хүсэлт илгээдэг. Сервер нь боловсруулсан өгөгдлийг илгээх замаар үйлчлүүлэгчид хариу үйлдэл үзүүлдэг.

```
channel request( int client, char data );  
channel response1( char res );  
channel response2( char res );
```

```
process Client1{  
  char values;  
  char results;  
  ... // Definition of values  
  send request( 1, values );  
  receive response1( results );  
  ... // Usage of results  
}  
process Client2{  
  char values;  
  char results;  
  ... // Definition of values  
  send request( 2, values );  
  receive response2( results );  
  ... // Usage of results  
}  
process Server{  
  int client;  
  char values;  
  char results;  
  ... // Initialisation  
  while true {  
    receive request( client, values );  
    if client = 1 then {  
      ... // Processing of values  
      send response1( results );  
    }  
    if cliente = 2 then {  
      ... // Processing of values  
      send response2( results );  
    }  
  }  
}
```

Дээрх жишээн дээр анхаарах хэд хэдэн зүйл байна. Нэгдүгээрт, хоёр үйлчлүүлэгч хоёулаа сервер рүү хүсэлт илгээхийн тулд сувгийн request ашигладаг. Харин сервер нь үйлчлүүлэгч бүрт нэг суваг ашиглан хариу илгээдэг. Цаашилбал, сувгийн нэр нь бүхэл бүтэн програмын хувьд статик,

глобал шинж чанартай байдаг тул харилцааны зөв байдал нь тэдгээрийн зөв ашиглалтаас хамаарна. Жишээлбэл, Client1 процесс нь хоёр дахь үйлчлүүлэгчийн хариултанд зориулагдсан channelresponse2-г ашиглаж болох бөгөөд үүнд ямар ч хяналт байхгүй. Боломжит алдаанаас зайлсхийхийн тулд, мөн нууцлалыг хадгалахын тулд үйлчлүүлэгч бүр сервер рүү илгээсэн мессеждээ өөрийн дотоод сувгийн нэрийг (бусад үйлчлүүлэгчдэд харагдахгүй) оруулахын тулд сервер хаашаа хариу илгээх ёстойг зааж өгч болно. Гэхдээ энэ нь динамик нэрийн менежментийг шаарддаг бөгөөд энэ нь ерөнхийдөө энгийн зүйл биш юм. Эцэст нь жишээг энгийн байх үүднээс хоёр үйлчлүүлэгчийн хувьд өгсөн боловч n сувгийн массивыг ашиглан статик байдлаар мэдэгдэж буй n үйлчлүүлэгчид үүнийг хялбархан өргөтгөж болно. Нөгөөтэйгүүр, өмнө нь мэдэгдээгүй хэд хэдэн үйлчлүүлэгчийг удирдах жишээг өргөтгөх нь илүү төвөгтэй байдаг (дахин хэлэхэд бид үйлчлүүлэгч бүр серверт сувгийнхаа нэрийг хэлэх боломжийг олгохын тулд динамик нэрийн менежмент хэрэгтэй болно).

14.5.3 Синхрон харилцаа

Синхрон харилцаанд (мэдээлэл солилцох) receive үйлдэл нь асинхрон тохиолдолтой адил байна. Гэсэн хэдий ч, send үйлдлийг блоклог байдлаар өөрчилдөг, өөрөөр хэлбэл илгээгч нь мессеж илгээсний дараа хүлээн авагч үүнийг хүлээн авах хүртэл блоклогддог. Тиймээс, наад зах нь онолын үүднээс авч үзвэл, синхрон тохиолдолд бид send, receive авах хоёр үйлдлийг харилцаа бодитоор хэрэгжих үед процессуудыг синхрончлох боломжийг олгодог гэж харж болно. Бодит хэрэглээний үүднээс авч үзвэл, үйлдлүүд өөр өөр цаг үед явагддаг бөгөөд илгээгч нь send явсны дараа мессежийг хүлээн авсныг баталгаажуулах дохиог хүлээнэ. Синхрон харилцаа нь анх CSP (communications service providers) дээр гарч ирсэн.

Синхрон харилцааны асинхроноос гол давуу тал нь бид сувагтай холбоотой мессежийн дарааллын хамгийн их уртыг хязгаарлаж чаддагт оршино. Үнэн хэрэгтээ, өмнөх мессежийг хүлээн авч дарааллаас хасах хүртэл ямар ч процесс өөр тийш нь мессеж илгээх боломжгүй. Тиймээс, дараалал нь илгээх процесс бүрт хамгийн ихдээ нэг мессеж агуулна. Синхрон харилцааны өөр нэг давуу тал нь алдаатай ажиллах явдал юм. Хэрэв мессеж хүлээн авахад алдаа гарсан бол, илгээгч нь харилцаа дуусахыг хүлээж гацсан бол илгээгч өөрийн тооцооллоо үргэлжлүүлснээс хэвийн бус нөхцөл байдлыг зохицуулах нь илүү хялбар байдаг.

Мэдээжийн хэрэг, синхрон харилцаа нь сул талуудтай. Нэгдүгээрт, send-ийг блоклож байгаатай холбоотойгоор процесс хоорондын зэрэгцээ байдал буурдаг: send, receive үйлдлүүд өөр өөр цаг үед явагддаг тул харилцаа хийгдэхэд үргэлж дор хаяж нэг блоклогдсон процесс байдаг. receive-аас өмнө send үйлдлийг гүйцэтгэх үед асинхрон холбооны хувьд энэ нь тийм биш юм. Энэ хязгаарлалт нь бодит хэрэглээний програмуудад хамааралтай байж болно. Зэрэгцээ харилцан үйлчлэлийн ердийн хэв маягийн дагуу бид өгөгдөл илгээдэг *үйлдвэрлэгч*, түүнийг хүлээн авдаг *хэрэглэгчийн* процесстэй байна гэж бодъё. Хэрэв хоёр процесс өөр өөр боловсруулалтын хугацаатай бол sends болон receives эрэмбээр гүйцэтгэгдэх баталгаа байхгүй болно. Ямар ч тохиолдолд синхрон харилцаа бүрт аль нэг процесс зогссоноос болж удаашралт (delay) гардаг. Нөгөө талаас, хэрэв бид асинхрон харилцааг ашигласан бол сувагтай холбоотой дараалал нь буфер болж, янз бүрийн хурдны зөрүүг багасгах тул нийт саатлыг багасгах болно: үйлдвэрлэгч нь хэрэглэгчээс хурдан байх үед мессежийн дараалал суваг дотор байна; эсрэг талдаа, хэрэглэгч боловсруулах боломжтой аль болох олон мессежийг (хэрэв байгаа бол) сонгож болно. Бид үйлдвэрлэгч ба хэрэглэгчийн хооронд буфер процессыг оруулснаар синхрон харилцаатай ижил төстэй ажиллагаа бий болгох боломжтой боловч энэ тохиолдолд ч гэсэн үр ашиг нь асинхрон хувилбараас бага байдаг.

Ерөнхийдөө үр ашгийг нь эс тооцвол бид асинхрон сүвгийг дамжуулж, сүвгийн илгээгч болон хүлээн авагчийн аль алинд нь шууд хариу үйлдэл үзүүлэх процессыг ашиглан синхрон харилцаагаар дамжуулан асинхрон холбоог дуурайж чадна. Үүний эсрэгээр мөн хийх боломжтой, учир нь бид блоклогддоггүй хүлээн авах процессоос ресепшиний мэдэгдэл хүлээдэг send-ийг receive-тэй үргэлж хослуулж болно, ийм байдлаар синхрон харилцааг бий болгодог.

Синхрон харилцааны хоёр дахь сул тал бол программист ашиглахад илүү төвөгтэй байдаг. Өөрөөр хэлбэл, программист илүү их анхаарал хандуулах ёстой, учир нь синхрон харилцааны нөлөөгөөр *түгжрэл* болон бусад алдааг гаргах нь илүү дөхөм болдог. Жишээ болгон sends болон receives үйлдлийг хоёуланг биелүүлэх хоёр процессыг авч үзье (энэ нь бараг бүх peer-to-peer програмуудад тохиолддог). Хэрэв хоёр процесс хоёулаа send гүйцэтгэвэл процесс бүр нөгөөгөөсөө receive-ыг хүлээж гацах тул тооцоолол цааш үргэлжлэх боломжгүй түгжрэлд орно. Ийм алдаа асинхрон холбооны үед гарахгүй, учир нь энэ тохиолдолд send үйлдэл нь блоклогддоггүй (мэдээжийн хэрэг асинхрон холболттой үед түгжрэл бусад хэлбэр үүсч болно).

14.5.4 Алсын процедур дуудалт ба Rendez-Vous

Одоо бид практикт, ялангуяа клиент-сервер харилцан үйлчлэлд өргөн хэрэглэгддэг харилцааны хоёр механизмыг авч үзье. Олон хэлээр (Java-г оруулаад) ашигладаг Remote Procedure Call (эсвэл RPC) болон Ада-д нэвтрүүлж, өргөн хэрэглэгддэг rendez-vous. Эхний төсөөллөөр бид эдгээр хоёр механизмыг мессеж солилцдог синхрон харилцааны арга гэж харж болно. Энд өмнөх догол мөрөнд харагдсан синхрон send-ээс ялгаатай нь илгээгч нь мессеж илгээсний дараа тооцооллыг үргэлжлүүлэхийн өмнө хүлээн авагчаас хариу мессеж ирэхийг (мессеж хүлээн авсан гэсэн мэдэгдэл биш) хүлээдэг. Илүү нарийвчлалтайгаар, эдгээр үйлдлүүдийн нэрнээс харахад RPC болон rendez-vous аль аль нь алсаас биелүүлдэг процедурыг ихэвчлэн өөр машин дээр ажилладаг, тэр ч байтугай газарзүйн хувьд алслагдсан процессоор гүйцэтгэдэг. RPC болон rendez-vous хоёрын ялгаа нь алсын дуудалт хийх аргад оршдог.

RPC-ийн хувьд хүлээн авагч нь тухайн дуудалтыг хүлээж байдаг идэвхтэй процесс байхгүй. Үүний эсрэгээр, хүлээн авагч мессежийг хүлээн авахдаа түүнийг боловсруулах шинэ процессыг бий болгодог. Энэ процесс нь параметруудийг дамжуулах, дуудагдсан процедурын их биеийг гүйцэтгэх, үр дүнг дуудагч руу дамжуулах үүрэгтэй. Үүнийг хийсний дараа процесс нь гүйцэтгэлээ зогсоох бөгөөд аливаа шинэ дуудалт нь өөр, тусгай зориулалтын боловсруулах процессыг бий болгох trigger болдог. Мэдээжийн хэрэг, харилцааны суурь бүтэц нь порт, суваг гэх мэт механизмаар угтагчийг зохицуулах, шинэ мессежийг угтах үед хүлээн авагчийг идэвхжүүлэх ажлыг хариуцах ёстой. Мессеж солилцооны хувьд RPC нь дуудалт гэж нэрлэгддэг зөвхөн илгээх ил үйлдлүүдийг гүйцэтгэдэг боловч хүлээн авах ил үйлдлүүдийг хийдэггүй гэж бид хэлж чадна.

Харин rendez-vous-ийн тусламжтайгаар дуудалтын процесс нь хүлээн авагчид идэвхтэй байдаг өмнө нь үүссэн (already-existing) процесс бүхий rendez-vous-ийг гүйцэтгэдэг. Дуудалт ирэх үед процесс нь хүссэн процедурыг гүйцэтгэдэг (параметруудийг дамжуулсны дараа ердийнх шиг), үр дүнг дуудагч руу дамжуулж, дараа нь үйл ажиллагаагаа үргэлжлүүлнэ, жишээлбэл, шинэ хүсэлтийг зохицуулах. Тиймээс, энэ тохиолдолд илгээгч талын call-аас гадна хүлээн авагч дээр 14.5.1-д өмнө нь харагдсан Ада дахь accept гэх мэт хүлээн авах ил үйлдэл байдаг. receive тохиолдолтой адилаар боловсруулагдах хүсэлт (call) байхгүй тохиолдолд accept-ыг түр хүлээлгэнэ.

Дараах хэсэгт бид RPC-ийн талаарх дэлгэрэнгүй мэдээллийг харж, сонирхсон уншигчдыг rendez-vous-ийн талаар тусгайлсан ном зохиолоос лавлана.

Процедурын алсын дуудалт хэрэгжүүлэхийн тулд процедурын кодыг агуулсан процесс нь дор хаяж процедурын нэрийг гадаад орчинд нийтлэх ёстой бөгөөд ингэснээр дуудалтад харагдах бөгөөд ашиглах боломжтой болно. Энэ нь ихэвчлэн модуль ашигладаг програмчлалын контекст дээр хийгддэг бөгөөд дараах тохиолдолд бид эдгээр бүтэцтэй байх болно гэж таамаглаж байна. Тиймээс модуль нь процесс болон экспортлох процедурыг агуулсан (програмын) бүрдэл хэсэг юм. Эдгээр нь модуль дотроо ил тодорхойлогдсон бөгөөд зөвхөн модулийн хүрээнд ашиглах боломжтой локал процедураас ялгаатай. Цаашилбал, модулийн процессууд нь background процесс гэж нэрлэгддэг цэвэр локал процессууд болон гаднах процедурын дуудалтуудад үйлчлэх зориулалттай үүсгэгдсэн процессуудад хуваагддаг. Өөр өөр модулиуд өөр өөр машинууд дээр байрлаж, алсын процедурыг дуудаж өөр хоорондоо холбогдож болно. Тиймээс модуль нь дараах бүтэцтэй байна.

```
module modName{
  op proc1( type of formal parameters ) returns ( type of result )
    // headers
  ...
  body {
    local variables;
    initialisation code;
    proc proc1( formal parameters ) returns ( result ){
      procedure body
    }      // implementation
    ...
    local procedures;
    local processes;
  }
}
```

Эндээс харахад эхний хэсэгт экспорт хийх боломжтой процедуруудын нэрсийг флоиад параметруудийн нэр, төрлүүд болон буцаах үр дүнгийн хамт жагсаасан болно. Дараа нь үндсэн хэсэгт, proc гэсэн түлхүүр үгээр бид процедур бүрийн хэрэгжилтийг тодорхойлдог (формал параметрууд ба үр дүнгийн төрлийг энд давтахгүй, зөвхөн нэрийг нь тэмдэглэнэ).

Дээрх кодыг иш татах өөр модуль нь modName модулийн proc1 процедурыг дуудахдаа эдгээр хоёр нэр болон одоогийн параметруудийг заасан call командайг ашиглана.

call modName.proc1(actual parameters)

Өмнө дурьдсанчлан ийм дуудалт нь одоогийн параметруудийг хүлээн авсны дараа proc1 процедурын үндсэн хэсгийг гүйцэтгэдэг шинэ процессыг бий болгодог. Энэ нь дуусмагц процесс нь дуудагч руу үр дүнг илгээж, дуусгавар болно. call гүйцэтгэсний дараа тээглэсэн дуудагч үр дүнг хүлээн авмагц гүйцэтгэлээ үргэлжлүүлж болно.

Хэрэв нэг модуль дахь янз бүрийн идэвхтэй процессууд - background ажиллаж байгаа болон алсын дуудлагыг гүйцэтгэж байгаа процессууд зэрэгцэн ажиллаж байвал бид RPC-ээс ямар ч аргаар хангаагүй тул тохиромжтой синхрончлол, харилцан хориглох механизмыг ашиглах ёстой гэдгийг

анхаарна уу. Өөрөөр хэлбэл, модуль бүрт монитортой адил ямар ч үед хамгийн ихдээ нэг идэвхтэй процесс байдаг гэж бид таамаглаж болно.

Эцэст нь RPC-г хэлээр шууд дэмждэггүй тохиолдолд бид RPC-тэй ажиллахад шаардлагатай харилцааны дэлгэрэнгүй мэдээллийг програмистаас далдалж, алсын процедур дуудалтыг "хэвийн" процедурын дуудлагатай аль болох адилхан болгохын тулд одоо байгаа хэлэнд тэдгээрийг *stub* гэж нэрлэн нэмж болохыг анхаарна уу. Stub-ийг ихэвчлэн *stub компилятор* автоматаар үүсгэдэг. Программ нь алсаас дуудлага хийх үед энэ нь үйлдлийг яг таг тодорхойлоход шаардлагатай бүх параметруудийг (дуудсан процедурын нэр, бодит параметрууд гэх мэт) авдаг stub дуудаж, тэдгээрийг агуулсан зохих мессежийг бүтээж, мессежийг хүлээн авагч дээр байрлах өөр stub (заримдаа *skeleton* гэж нэрлэдэг)-руу илгээдэг. Хоёрдахь stub нь мессежийг хүлээн авч, параметруудийг задалж, дараа нь шинэ процесс үүсгэсэн локал процедурыг дууддаг. Гүйцэтгэх ажиллагаа дуусмагц эхний харилцан үйлчлэлийн талаар давхар байдлаар, хоёр дахь stub нь үр дүнгийн хамт мессеж үүсгэж, эхний stub руу дамжуулж, эцэст нь үр дүнг гаргаж аваад RPC-г дуудсан процесс руу дамжуулдаг.

14.5.5

14.5.6

14.5.7 й